

Assignment2

Hooman Fareghi

2026-02-12

```
knitr::opts_chunk$set(echo = TRUE)
```

```
#preparing data  
library(caret)
```

```
## Loading required package: ggplot2
```

```
## Loading required package: lattice
```

```
set.seed(123)  
#remove space from names  
Bank <- read.csv("UniversalBank-1.csv", check.names = TRUE)  
names(Bank) <- make.names(names(Bank))  
#targeting personal loans  
Bank$Personal.Loan <- factor(Bank$Personal.Loan, levels = c(0,1), labels = c("0", "1"))  
#excluding ID and zip code(Perform a k-NN classification with all predictors except ID and ZIP code)  
Bank2 <- Bank[, -c(1,5)]  
#transform categorical predictors with more than two categories into dummy variables first.  
Bank2$Family <- factor(Bank2$Family)  
Bank2$Education <- factor(Bank2$Education, levels = c(1,2,3))  
Bank3 <- as.data.frame(model.matrix(~.-Personal.Loan, data=Bank2))  
Value <- Bank2$Personal.Loan  
#Partition the data into training (60%) and validation (40%) sets  
partition <- createDataPartition(Value, p=0.6, list = FALSE)  
x_train <- Bank3[partition,]  
x_test <- Bank3[-partition,]  
y_train <- Value[partition]  
y_test <- Value[-partition]  
#Perform a k-NN classification with all predictors  
pp <- preProcess(x_train, method = c("center", "scale"))
```

```
## Warning in preProcess.default(x_train, method = c("center", "scale")): These  
## variables have zero variances: (Intercept)
```

```
x_train2 <- predict(pp, x_train)  
x_test2 <- predict(pp, x_test)  
#check  
dim(x_train2); dim(x_test2)
```

```
## [1] 3000 15
```

```
## [1] 2000 15
```

```
table(y_train);table(y_test)
```

```
## y_train
##    0    1
## 2712 288
```

```
## y_test
##    0    1
## 1808 192
```

#1. Age = 40, Experience = 10, Income = 84, Family = 2, CCAvg = 2, Education_1 = 0, Education_2 = 1, Education_3 = 0, Mortgage = 0, Securities Account = 0, CD Account = 0, Online = 1, and Credit Card = 1. Perform a k-NN classification with all predictors except ID and ZIP code using k = 1. Remember to transform categorical predictors with more than two categories into dummy variables first. Specify the success class as 1 (loan acceptance), and use the default cutoff value of 0.5. How would this customer be classified?

```
knn <- knn3(x=x_train2,y=y_train,k=1)
customer1 <- data.frame(Age=40, Experience=10,Income=84,Family=factor("2",levels = levels(Bank2$Family)),
customer1
```

```
##   Age Experience Income Family CCAvg Education Mortgage Securities.Account
## 1  40          10     84      2      2          1          0              0
##   CD.Account Online Credit.Card Personal.Loan
## 1          0      1          1              0
```

```
customer1_x <- as.data.frame(model.matrix(~.-Personal.Loan,data = customer1))
missing_cols <- setdiff(colnames(x_train), colnames(customer1_x))
for(m in missing_cols) customer1_x[[m]] <- 0

customer1_x <- customer1_x[, colnames(x_train), drop = FALSE]

customer1_x2 <- predict(pp,customer1_x)
p1 <- predict(knn , customer1_x2, type = "prob")[, "1"]
predict1 <- ifelse(p1>=0.5,"1","0")
p1
```

```
## 1
## 0
```

```
predict1
```

```
## 1
## "0"
```

#k=1 and a cutoff of 0.5, the predicted probability that Customer1 accepts the loan is 0. so the model shows that the customer1 as not accepting the loan.

#2. What is a choice of k that balances between overfitting and ignoring the predictor information?

```

k_grid <- seq(1, 101, by = 2)
acc <- numeric(length(k_grid))
for(i in seq_along(k_grid)){k <- k_grid[i]
fit <- knn3(x = x_train2, y = y_train, k = k)
p <- predict(fit, x_test2, type = "prob")[, "1"]
pred <- factor(ifelse(p >= 0.5, "1", "0"), levels = c("0", "1"))
acc[i] <- mean(pred == y_test)}
tuning <- data.frame(k = k_grid, validation_accuracy = acc)
tuning

```

```

##      k validation_accuracy
## 1      1          0.9640
## 2      3          0.9605
## 3      5          0.9580
## 4      7          0.9495
## 5      9          0.9410
## 6     11          0.9390
## 7     13          0.9400
## 8     15          0.9355
## 9     17          0.9330
## 10    19          0.9330
## 11    21          0.9300
## 12    23          0.9300
## 13    25          0.9285
## 14    27          0.9280
## 15    29          0.9275
## 16    31          0.9275
## 17    33          0.9260
## 18    35          0.9245
## 19    37          0.9265
## 20    39          0.9240
## 21    41          0.9250
## 22    43          0.9230
## 23    45          0.9225
## 24    47          0.9215
## 25    49          0.9205
## 26    51          0.9215
## 27    53          0.9210
## 28    55          0.9205
## 29    57          0.9205
## 30    59          0.9185
## 31    61          0.9190
## 32    63          0.9180
## 33    65          0.9180
## 34    67          0.9180
## 35    69          0.9180
## 36    71          0.9175
## 37    73          0.9175
## 38    75          0.9170
## 39    77          0.9160
## 40    79          0.9155
## 41    81          0.9155
## 42    83          0.9155

```

```
## 43 85          0.9155
## 44 87          0.9155
## 45 89          0.9155
## 46 91          0.9135
## 47 93          0.9130
## 48 95          0.9130
## 49 97          0.9130
## 50 99          0.9130
## 51 101         0.9115
```

```
best_k <- tuning$k[which.max(tuning$validation_accuracy)]
best_k
```

```
## [1] 1
```

```
max(tuning$validation_accuracy)
```

```
## [1] 0.964
```

#I tested different values of k from 1 to 101. The highest validation accuracy (0.964) was achieved at k = 1. Although small k values can overfit, k = 1 gave the best validation result in this dataset. Therefore, k = 1 was selected.

#3. Show the confusion matrix for the validation data that results from using the best k.

```
knn_best <- knn3(x = x_train2, y = y_train, k = best_k)
p_valid <- predict(knn_best, x_test2, type = "prob")[, "1"]
pred_valid <- factor(ifelse(p_valid >= 0.5, "1", "0"), levels = c("0", "1"))

confusionMatrix(pred_valid, y_test, positive = "1")
```

```
## Confusion Matrix and Statistics
```

```
##
##           Reference
## Prediction    0    1
##           0 1793   57
##           1   15  135
##
##           Accuracy : 0.964
##           95% CI : (0.9549, 0.9717)
##           No Information Rate : 0.904
##           P-Value [Acc > NIR] : < 2.2e-16
##
##           Kappa : 0.7701
##
##           McNemar's Test P-Value : 1.352e-06
##
##           Sensitivity : 0.7031
##           Specificity : 0.9917
##           Pos Pred Value : 0.9000
##           Neg Pred Value : 0.9692
##           Prevalence : 0.0960
```

```
##          Detection Rate : 0.0675
##    Detection Prevalence : 0.0750
##      Balanced Accuracy : 0.8474
##
##      'Positive' Class : 1
##
```

#The model has 96.4% accuracy. It predicts non-loan customers very well (99%) and correctly identifies about 70% of loan customers. It performs better for the majority class (no loan).

#4. Consider the following customer: Age = 40, Experience = 10, Income = 84, Family = 2, CCAvg = 2, Education_1 = 0, Education_2 = 1, Education_3 = 0, Mortgage = 0, Securities Account = 0, CD Account = 0, Online = 1 and Credit Card = 1. Classify the customer using the best k.

```
customer2 <- data.frame(Age = 40, Experience = 10, Income = 84,
  Family = factor("2", levels = levels(Bank2$Family)), CCAvg = 2, Education = factor("3", levels = levels(
  Online = 1, CreditCard = 1, Personal.Loan = factor("0", levels = c("0", "1")))
customer2
```

```
##   Age Experience Income Family CCAvg Education Mortgage Securities.Account
## 1  40         10     84      2      2          3          0              0
##   CD.Account Online CreditCard Personal.Loan
## 1          0          1          1          0
```

```
customer2_X <- as.data.frame(model.matrix(~ . - Personal.Loan, data = customer2))

missing_cols <- setdiff(colnames(x_train), colnames(customer2_X))
for(m in missing_cols) customer2_X[[m]] <- 0
customer2_X <- customer2_X[, colnames(x_train), drop = FALSE]

customer2s <- predict(pp, customer2_X)

p2 <- predict(knn_best, customer2s, type = "prob")[, "1"]
pred2 <- ifelse(p2 >= 0.5, "1", "0")

p2
```

```
## 1
## 0
```

```
pred2
```

```
## 1
## "0"
```

```
acc[i] <- mean(pred2 == y_test)

tuning2 <- data.frame(k = k_grid, validation_accuracy = acc)
tuning
```

```
##      k validation_accuracy
```

## 1	1	0.9640
## 2	3	0.9605
## 3	5	0.9580
## 4	7	0.9495
## 5	9	0.9410
## 6	11	0.9390
## 7	13	0.9400
## 8	15	0.9355
## 9	17	0.9330
## 10	19	0.9330
## 11	21	0.9300
## 12	23	0.9300
## 13	25	0.9285
## 14	27	0.9280
## 15	29	0.9275
## 16	31	0.9275
## 17	33	0.9260
## 18	35	0.9245
## 19	37	0.9265
## 20	39	0.9240
## 21	41	0.9250
## 22	43	0.9230
## 23	45	0.9225
## 24	47	0.9215
## 25	49	0.9205
## 26	51	0.9215
## 27	53	0.9210
## 28	55	0.9205
## 29	57	0.9205
## 30	59	0.9185
## 31	61	0.9190
## 32	63	0.9180
## 33	65	0.9180
## 34	67	0.9180
## 35	69	0.9180
## 36	71	0.9175
## 37	73	0.9175
## 38	75	0.9170
## 39	77	0.9160
## 40	79	0.9155
## 41	81	0.9155
## 42	83	0.9155
## 43	85	0.9155
## 44	87	0.9155
## 45	89	0.9155
## 46	91	0.9135
## 47	93	0.9130
## 48	95	0.9130
## 49	97	0.9130
## 50	99	0.9130
## 51	101	0.9115

```
best_k2 <- tuning2$k[which.max(tuning$validation_accuracy)]
best_k2
```

```
## [1] 1
```

```
max(tuning2$validation_accuracy)
```

```
## [1] 0.964
```

#Changing Education did not change the prediction because with $k = 1$ the model depends on only one nearest neighbor.

#5. Repartition the data, this me into training, validation, and test sets (50% : 30% : 20%). Apply the k-NN method with the k chosen above. Compare the confusion matrix of the test set with that of the training and validation sets. Comment on the differences and their reason.

```
library(caret)
set.seed(456)

#splitting data to %
partition2 <- createDataPartition(Value, p = 0.50, list = FALSE)
X_train2 <- Bank3[partition2, ]
y_train2 <- Value[partition2]

# remaining 50%
X_temp <- Bank3[-partition2, ]
y_temp <- Value[-partition2]

# split remaining 30% to validation and 20% to test
idx_valid2 <- createDataPartition(y_temp, p = 0.60, list = FALSE)
X_valid2 <- X_temp[idx_valid2, ]
y_valid2 <- y_temp[idx_valid2]
X_test2 <- X_temp[-idx_valid2, ]
y_test2 <- y_temp[-idx_valid2]

# standardize variables
pp2 <- preProcess(X_train2, method = c("center", "scale"))
```

```
## Warning in preProcess.default(X_train2, method = c("center", "scale")): These
## variables have zero variances: (Intercept)
```

```
X_train2_s <- predict(pp2, X_train2)
X_valid2_s <- predict(pp2, X_valid2)
X_test2_s <- predict(pp2, X_test2)

# kNN using the SAME best_k from q3
knn_final <- knn3(x = X_train2_s, y = y_train2, k = best_k)

# Training confusion matrix
p_tr <- predict(knn_final, X_train2_s, type = "prob")[, "1"]
pred_tr <- factor(ifelse(p_tr >= 0.5, "1", "0"), levels = c("0", "1"))
cm_train <- confusionMatrix(pred_tr, y_train2, positive = "1")
cm_train
```

```
## Confusion Matrix and Statistics
```

```
##
##           Reference
## Prediction    0    1
##           0 2260    0
##           1    0  240
##
##           Accuracy : 1
##           95% CI : (0.9985, 1)
##       No Information Rate : 0.904
##       P-Value [Acc > NIR] : < 2.2e-16
##
##           Kappa : 1
##
## Mcnemar's Test P-Value : NA
##
##           Sensitivity : 1.000
##           Specificity : 1.000
##       Pos Pred Value : 1.000
##       Neg Pred Value : 1.000
##           Prevalence : 0.096
##       Detection Rate : 0.096
##       Detection Prevalence : 0.096
##       Balanced Accuracy : 1.000
##
##       'Positive' Class : 1
##
```

Validation confusion matrix

```
p_va <- predict(knn_final, X_valid2_s, type = "prob")[, "1"]
pred_va <- factor(ifelse(p_va >= 0.5, "1", "0"), levels = c("0", "1"))
cm_valid <- confusionMatrix(pred_va, y_valid2, positive = "1")
cm_valid
```

```
## Confusion Matrix and Statistics
##
##           Reference
## Prediction    0    1
##           0 1339   47
##           1   17   97
##
##           Accuracy : 0.9573
##           95% CI : (0.9458, 0.967)
##       No Information Rate : 0.904
##       P-Value [Acc > NIR] : 5.372e-15
##
##           Kappa : 0.7289
##
## Mcnemar's Test P-Value : 0.000289
##
##           Sensitivity : 0.67361
##           Specificity : 0.98746
##       Pos Pred Value : 0.85088
##       Neg Pred Value : 0.96609
##           Prevalence : 0.09600
```



```
##          Detection Rate : 0.06467
##    Detection Prevalence : 0.07600
##      Balanced Accuracy : 0.83054
##
##      'Positive' Class : 1
##
```

```
# Test confusion matrix
```

```
p_te <- predict(knn_final, X_test2_s, type = "prob")[, "1"]
pred_te <- factor(ifelse(p_te >= 0.5, "1", "0"), levels = c("0", "1"))
cm_test <- confusionMatrix(pred_te, y_test2, positive = "1")
cm_test
```

```
## Confusion Matrix and Statistics
```

```
##
##          Reference
## Prediction  0   1
##          0 897  35
##          1   7  61
##
##          Accuracy : 0.958
##          95% CI : (0.9436, 0.9696)
##    No Information Rate : 0.904
##    P-Value [Acc > NIR] : 9.200e-11
##
##          Kappa : 0.7218
##
##    McNemar's Test P-Value : 3.097e-05
##
##          Sensitivity : 0.6354
##          Specificity : 0.9923
##          Pos Pred Value : 0.8971
##          Neg Pred Value : 0.9624
##          Prevalence : 0.0960
##          Detection Rate : 0.0610
##    Detection Prevalence : 0.0680
##          Balanced Accuracy : 0.8138
##
##      'Positive' Class : 1
##
```

#Using the 50/30/20 split, the training accuracy is 100% because $k = 1$ perfectly fits the training data. The validation accuracy is 95.7% and the test accuracy is 95.8%, which are very similar. This shows that the model performs consistently on new data. The model predicts non-loan customers very well, but sensitivity for loan customers is lower.